X-Rite Driver System v4

(XDS 4)

Technical Specification

Version 1.2
9 May 2014

X-Rite Driver System 4 – Technical Spec

Table of Contents

1 Introduction 5
1.1 Overview 5
1.2 Supported Instruments 5
1.3 Deliverables 5
2 Architecture 6
2.1 Driver Package 6
2.2 Command Structure 6
2.3 Installer 6
2.4 File Structure 6
2.4.1 Drivers 6
2.4.2 Driver Data 6
2.4.3 SPECTRO.INI 7
2.5 User Interface 7
2.6 Driver Interface 7
2.7 Test Application and Sample Code 7
2.8 Logging 7
2.9 Operating System Support 8
2.10 Application Sharing 8
2.11 Instrument Settings 8
2.12 Calibration 8
3 Driver Interface 9
3.1 Basic Operation 9
3.1.1 GetInterfaceVersion 9
3.1.2 Connect 9
3.1.3 Disconnect 10
3.1.4 IsConnected 10
3.1.5 GetCalibrationStandard 11
3.1.6 GetSerialNum 12
3.1.7 GetSpectralSetCount 12
3.1.8 GetWavelengthCount 13
3.1.9 GetSpectralSetName 13

3.1.10 GetWavelengthValue 14
3.2 Measurement 14
3.2.1 Measure 14
3.2.2 GetSpectralData 15
3.2.3 IsDataReady 15
3.3 Calibration 16
3.3.1 GetCalStatus 16
3.3.2 GetCalSteps 16
3.3.3 CalibrateStep 18
3.3.4 GetCalMode 18
3.3.5 GetCalProgress 19
3.3.6 AbortCalibration 20
3.4 Stored Samples 20
3.4.1 ClearSamples 20
3.4.2 GetSampleCount 20
3.4.3 SetCurrentSample 21
3.4.4 GetSampleData 21
3.5 Settings 22
3.5.1 GetAvailableSettings 22
3.5.2 GetSettingOptions 22
3.5.3 GetOption 23
3.5.4 SetOption 23
3.5.5 Settings Table 24
3.6 Scanning Measurement 26
3.6.1 ScanIsSupported 26
3.6.2 ScanConfig 26
3.6.3 ScanStart 27
3.6.4 ScanAbort 27
3.6.5 ScanGetStatus 28
3.6.6 ScanGetCount 29
3.6.7 ScanGetData 29
3.7 Error Handling 30

3.7.1 GetLastErrorCode 30
3.7.2 GetLastErrorString 30
3.8 Pass-Through or Custom Commands 31
3.8.1 Execute 31
3.8.2 ExecuteBinary 31
4 Error Codes 33
4.1.1 General Errors 33
4.1.2 Instrument Specific Errors 33

X-Rite Driver System 4 – Technical Spec

1 Introduction

1.1 Overview

The X-Rite Driver System v4 (XDS 4) provides a set of drivers with a common interface for many X-Rite spectrophotometers. It will be used in X-Rite software applications and in third-party software applications. It will contain high level drivers (unlike XDS III) and as a set of simple Windows DLLs will be accessible to all programming languages.

1.2 Supported Instruments

Instruments

- 552
- 939
- 962/964
- CF57
- CFS57
- Ci4100/Ci4200
- Ci51/Ci52
- Ci61/Ci62/Ci64
- eXact
- MA96
- RM400
- RM425
- RM61
- SP52
- SP62/SP64
- VS205
- VS450

Note: XDS 4 supports a variety of X-Rite instruments. This document might reference instruments that are not available to a particular licensee, because their license agreement does not include those instruments.

1.3 Deliverables

The XDS 4 driver package includes the following items:

- Header file (.h)
- One DLL for each type of instrument
- `SPECTRO.INI` file that lists each instrument and the name of the corresponding driver DLL
- Source and executable code for a sample application, which shows how to connect, calibrate, and measure with each instrument
- Document of error codes and strings

2 Architecture

2.1 Driver Package

- The driver package consists of a DLL for each instrument plus a `SPECTRO.INI` file that lists each instrument and the name of the DLL.
- This allows each application to support a particular set of instruments and to read the `SPECTRO.INI` file to determine which instruments it supports.

2.2 Command Structure

- The drivers provide a high level interface, with commands such as connect, measure, and calibrate.
- As a fallback, the drivers include the "Execute" and "ExecuteBinary" command, which can be used to send any arbitrary command. Each driver can implement any commands desired; if the application sends a command the driver does not implement, it merely passes the command to the instrument.

2.3 Installer

- The driver package does not include an installer.
- Each client application must install the drivers.

2.4 File Structure

2.4.1 Drivers

- Each application must place the drivers in an application-specific location. With this approach, there is no collision between applications.
- The driver file structure must be as follows:

```
ProgramFiles\
      <app>\
            SPECTRO.INI
            CI51.DLL
            VS205.DLL
            …
```

2.4.2 Driver Data

- The application must provide an application-specific place for persistent driver settings and log data.
- The application installer must grant read/write rights for this data folder.
- Each driver DLL uses its corresponding INI file in this folder to persist settings.
- The format of these driver INI files is internal; they must not be modified by the application.
- The spectro.log file is the log file produced by the various driver DLLs. See section 2.8, "Logging" for more information.

```
ProgramData\
      <path>\
            CI51.INI
```

```
VS205.INI
SPECTRO.LOG
```

2.4.3 SPECTRO.INI

- The `SPECTRO.INI` file contains information about the drivers, driver data, and other settings, such as debug level.

File Sections and Properties	Explanation
`[Options]`	
`Debug=0`	0 – set logging to normal level 1 – set logging to debug level
`[Data]`	
`Path=<path>`	Path to persisted settings and log files. Can be absolute path or relative path. A relative path starts in the all users data folder (e.g. ProgramData on Windows 7)
`[Drivers]`	
`Available=Ci51;VS205`	List of available drivers
`Ci51=CI51.DLL`	Driver DLL for Ci51 instrument
`VS205=VS205.DLL`	Driver DLL for VS205 instrument
…	…

2.5 User Interface

- The drivers are designed to be silent, such that they never display any user interface. The application is responsible for all UI, including configuration, measurement workflow, calibration workflow, and error messages.

2.6 Driver Interface

- To maximize ease of use by a wide variety of programming languages, the drivers will return only primitive types.
- No exceptions are thrown.

2.7 Test Application and Sample Code

- A simple test application will be provided with the drivers.
- Source and executable code will be provided.
- The test application lets the user choose an instrument, connect, calibrate, measure, and type an arbitrary command to send to the instrument (through the Execute command)
- Because source code is provided, the developer can see how to use our instruments for basic functions.

2.8 Logging

- Each driver writes logging information to `SPECTRO.LOG`. Each application will have its own log file (see following section).
- There are two logging levels:

    - Normal logging – logs only errors
    - Debug logging – shows every method call, date/time, response
- To enable debug logging, set logging level in `SPECTRO.INI`
- The log file contents will roll when it reaches approximately 100K. Half the contents are replaced at a time.

2.9 Operating System Support

- Supported operating systems
    - Windows XP (32 or 64 bit)
    - Windows Vista (32 or 64 bit)
    - Windows 7 (32 or 64 bit)
    - Windows 8 (32 or 64 bit)
- All driver methods operate without requiring administrator rights, including settings and logging.
- All data files are stored in the ProgramData location (not Program Files). See section 2.4, "File Structure" for details.
- 32 and 64 bit support
    - Two sets of DLLs (same source code, compiled differently)
        - 32-bit (32-bit apps must use 32-bit DLLs)
        - 64-bit (64-bit apps must use 64-bit DLLs)
    - Two sets of kernel level drivers
        - 32-bit drivers
        - 64-bit drivers

2.10 Application Sharing

- If two or more applications need to share an instrument, each application needs to use Connect() and Disconnect() as appropriate to let other applications communicate with the instrument

2.11 Instrument Settings

- Instrument settings are handled through various settings methods. See section "3.5 Settings" for details.
- The driver will persist the settings in an application-specific and device-specific INI file (e.g. CI51.INI, VS205.INI), so the calling application does not need to do so. Every time Connect() is called the instrument will be configured according to the last persisted settings.
- The persistent storage will be done in such a way that administrator rights are not needed.

2.12 Calibration

Instrument calibration is handled as follows:

- The application uses the silent calibration commands and displays its own user interface:

- GetCalibrationSteps()
- CalibrateStep(char *)

3 Driver Interface

3.1 Basic Operation

3.1.1 GetInterfaceVersion

Method Signature

- `const char* _stdcall GetInterfaceVersion();`

Description

- Describes the XDS4 version supported by instrument DLL.

Input Parameters

- None

Return Value

- XDS4 version. For example, instrument DLL's built with the XDS 4 version 1.2 will return "1.2".

Notes

- This method is new to XDS4 version 1.2 and not available in previous versions.

3.1.2 Connect

Method Signature

- `BOOL _stdcall Connect();`

Description

- Connect to the instrument

Input Parameters

- None

Return Value

- true – connection succeeded
- false – connection failed

Notes

- If the connection fails, the GetLastErrorCode() and GetLastErrorString() methods can be useful.

- See section 3.5.4, "SetOption" for information about the interaction between connections and settings.
- Some applications might want to disconnect after each instrument operation and then connect again before the next operation. This approach may be used *but only within certain guidelines:*
    - The application must *not* disconnect in the middle of a calibration. The entire calibration – meaning all calibration steps – must be performed without disconnecting.
    - The application must *not* disconnect in the middle of a measurement. The entire measurement – including getting the response from the instrument – must be performed without disconnecting.

3.1.3 Disconnect

Method Signature

- `BOOL _stdcall Disconnect();`

Description

- Close the connection to the instrument.

Input Parameters

- None

Return Value

- true – disconnect succeeded
- false – disconnect failed

Notes

- If the connection fails, the GetLastErrorCode() and GetLastErrorString() methods can be useful.

3.1.4 IsConnected

Method Signature

- `BOOL _stdcall IsConnected();`

Description

- Indicates whether the instrument is connected.

Input Parameters

- None

Return Value

- true – instrument is connected
- false – instrument is not connected

Notes

- None

3.1.5 GetCalibrationStandard

Method Signature

- `const char* _stdcall GetCalibrationStandard();`

Description

- Gets the calibration standard for the instrument.

Input Parameters

- None

Return Value

- A string representation of the calibration standard. See table below.

Instrument	Calibration Standard
552	Other
939	XRDI or XRGA
962	Other
964	Other
CF57	Other
CFS57	Other
Ci4100	Other
Ci4200	Other
Ci4200UV	Other
Ci5	Other
Ci51	Other
Ci51S	Other
Ci52	Other
Ci61	Other
CI61+RTL	Other
Ci62	Other
Ci64	Other
Ci64UV	Other
Ci7	Other
eXact	XRGA
MA96	Other
RM400	Other
RM425	Other
RM61	Other
SP52	Other
SP62	Other

SP64	Other
VS205	Other
VS450	Other

Notes

- The XDS 4 drivers do not perform any transforms from one calibration standard to another. The drivers merely determine the calibration standard of the connected instrument and report it to the calling application.

3.1.6 GetSerialNum

Method Signature

- `const char* _stdcall GetSerialNum();`

Description

- Gets the serial number of the instrument

Input Parameters

- None

Return Value

- Serial number of the instrument

Notes

- With some older instruments (e.g. CF57, CFS57), the serial number returned does not match the finished goods serial number printed on the outside of the instrument. In such cases, the serial number returned is that of the internal optics package.

3.1.7 GetSpectralSetCount

Method Signature

- `int _stdcall GetSpectralSetCount();`

Description

- Get the number of spectral sets supported by the instrument. An instrument might provide multiple spectral sets from a single measurement. The spectral sets can represent different measurement geometries (e.g. SPIN and SPEX) or measurement conditions (e.g. M0, M1, M2, and M3).

Input Parameters

- None

Return Value

- The number of spectral sets.

Notes

- A 0/45 instrument (e.g. VS205) will return 1.
- A d/8 instrument (e.g. SP64) will typically return 2, representing SPIN and SPEX.
- A graphic arts instrument that supports the ISO 13655-2009 standard may return up to 4, representing M0, M1, M2, and M3.
- This value is useful with GetSpectralSetName() and GetSpectralData().
- This method replaces GetAngles().

3.1.8 GetWavelengthCount

Method Signature

- `int _stdcall GetWavelengthCount();`

Description

- Get the number of spectral wavelengths that will be provided in measurement data.

Input Parameters

- None

Return Value

- The number of spectral wavelengths.

Notes

- A 16-point instrument (e.g. CF57) will return 16.
- A 31-point instrument (e.g. Ci51) will return 31.
- This value is useful with GetWavelengthValue() and GetSpectralData().
- This method replaces GetWavelengths().

3.1.9 GetSpectralSetName

Method Signature

- `const char* _stdcall GetSpectralSetName(int spectralset);`

Description

- Get the name of a spectral set.

Input Parameters

- spectralset – index of the spectral set (zero-based)

Return Value

- Name of the spectral set

Notes

- A 0/45 or 45/0 instrument will return “45”.
- A d/8 instrument will return “SPIN” and “SPEX”.

- An instrument that supports the ISO 13655-2009 standard will return one or more of the following: “M0”, “M1”, “M2”, and “M3”.
- This method replaces GetAngleValue().

3.1.10 GetWavelengthValue

Method Signature

- `int _stdcall GetWavelengthValue(int wl);`

Description

- Get the value of a wavelength, in nanometers.

Input Parameters

- wl – index of the wavelength (zero based)

Return Value

- Value of the wavelength, in nanometers

Notes

- An instrument that returns 31 spectral points, from 400-700 nm, would return values as follows:

Wavelength Index	Return Value
0	400
1	410
2	420
…	…
30	700

3.2 Measurement

3.2.1 Measure

Method Signature

- `BOOL _stdcall Measure();`

Description

- Tells the driver to initiate a measurement

Input Parameters

- None

Return Value

- true – measurement was taken
- false – measurement was not taken

Notes

- None

3.2.2 GetSpectralData

Method Signature

- `float _stdcall GetSpectralData(int spectralset, int wl);`

Description

- Returns spectral data for a given spectral set and wavelength

Input Parameters

- spectralset – index of the desired spectral set (zero based)
- wl – index of the desired wavelength (zero based)

Return Value

- Spectral data for the given spectral set and wavelength

Notes

- 100% reflectance is returned as 100.00

3.2.3 IsDataReady

Method Signature

- `BOOL _stdcall IsDataReady();`

Description

- Indicates whether measurement data is ready, meaning a measurement was completed

Input Parameters

- None

Return Value

- true – measurement data is ready
- false – no measurement data is ready

Notes

- If a measurement is triggered on the device, not by the software, the application should poll the driver using this command, to determine when the measurement is complete and the data is ready.
- If a measurement is triggered by software, this command is not needed.
- The IsDataReady() flag is reset after GetSpectralData() is called. Merely checking this flag does not reset it.

3.3 Calibration

3.3.1 GetCalStatus

Method Signature

- `int _stdcall GetCalStatus();`

Description

- returns the current calibration status of the instrument

Input Parameters

- None

Return Value

- 0 – calibrated
- 1 – not calibrated

Notes

- None

3.3.2 GetCalSteps

Method Signature

- `const char* _stdcall GetCalSteps();`

Description

- Returns the list of calibration steps for the instrument

Input Parameters

- None

Return Value

- A semi-colon separated string that contains the list of calibration steps in the required order.
- This list of calibration steps is instrument specific. The following table shows the return value for each instrument.
- The list of calibration steps for a given instrument can vary from time to time. For example, some instruments may or may not require black calibration, depending on

Calibration Steps

Instrument	Calibration Steps
552	“DeviceDisplay”
939	“DeviceDisplay”
962	“DeviceDisplay”
964	“DeviceDisplay”

CF57	“White;Black”
CFS57	“White;Black”
Ci4100	“White;Black”
Ci4200	“White;Black”
Ci4200UV	Same as Ci4200 if only White/Black needed. Otherwise, “White;Black;UV1;White;Black;UV2”
Ci5	“Black;White”
Ci51	“White;Black”
Ci52	“White;Black”
Ci61	“DeviceDisplay”
Ci61+RTL	“DeviceDisplay”
Ci62	“DeviceDisplay”
Ci64	“DeviceDisplay”
Ci64UV	“DeviceDisplay”
Ci7	“Black;White”
eXact	“Auto”
MA96	“DeviceDisplay”
RM400	“White”
RM425	“White”
RM61	“White;Black”
SP52	“White;Black”
SP62	“DeviceDisplay”
SP64	“DeviceDisplay”
VS205	“White”
VS450	“White”

Explanation of Calibration Steps

Calibration Step	Explanation
White	A white tile
Black	A black trap or open port
Auto	The instrument can be calibrated without any user interaction on the device. Specifically, it means the instrument can position the required calibration media automatically.
DeviceDisplay	The device display will tell the end user what is required for each step in the calibration process.
UV1	UV adjustment tile (to perform UV calibration)
UV2	UV adjustment tile (to verify UV calibration)

3.3.3 CalibrateStep

Method Signature

- `BOOL _stdcall CalibrateStep(const char* calstep);`

Description

- Perform the specified calibration step without displaying any user interface.

Input Parameters

- calstep – name of calibration step

Return Value

- true – calibration succeeded
- false – calibration failed

Notes

- The application should first call GetCalibrationSteps() and then call this method with each step.
- The application is responsible to display the appropriate instructions to the end user (e.g. present white tile, present black trap).
- The calibration steps <u>must</u> be performed in the order they are returned by GetCalibrationSteps().
- If the calibration mode is “Software”, this method will return after the instrument has completed the calibration step. If the calibration mode is “Device”, this method will return immediately.

3.3.4 GetCalMode

Method Signature

- `const char* _stdcall GetCalMode();`

Description

- Returns the current calibration mode, which indicates how calibration measurements are initiated

Input Parameters

- None

Return Value

- A string that describes how calibration measurements are initiated
- "Device" – the user initiates measurement of the calibration substrate (e.g. white tile, black trap) on the device
- "Software" – the user initiates measurement of the calibration substrate via the software application

Notes

- The calibration mode may vary depending on the current Reading_Mode of the device, however the calibration mode can be different than the Reading_Mode. (For example, the Reading_Mode might be "Software", yet the calibration mode could be "Device".) This is driven by the underlying instrument behavior.
- When this method returns "Device", the calling application should prompt the user to initiate each calibration step on the device.
- When this method returns "Software", the calling application should prompt the user to click a button in the software for each calibration step.

3.3.5 GetCalProgress

Method Signature

- `int _stdcall GetCalProgress();`

Description

- Returns the progress of a calibration process

Input Parameters

- None

Return Value

- 0 – Completed successfully
- 1 – Calibration still in progress
- Any other value – an error occurred (see Section 4, "Error Codes")

Notes

- Calibration can be a multi-step process, and it relies on the end user to take some physical action with the instrument (e.g. present white tile, present black trap). This method is useful to determine whether a calibration process is finished.

- This method is not useful with the DeviceDisplay calibration step. In that case, the user needs to tell the software application when they have completed calibration on the device.

3.3.6 AbortCalibration

Method Signature

- `BOOL _stdcall AbortCalibration();`

Description

- Cancels the current calibration process

Input Parameters

- None

Return Value

- true – successfully aborted calibration
- false – could not abort calibration

Notes

- This method can be used to allow the end user to cancel an in-progress calibration process. Specifically, it is used to cancel a CalibrateStep() operation when the software is waiting for the user to initiate a calibration step on the device.

3.4 Stored Samples

3.4.1 ClearSamples

Method Signature

- `BOOL _stdcall ClearSamples();`

Description

- Clears all stored samples from the instrument.

Input Parameters

- None

Return Value

- true – successfully cleared stored samples
- false – failed to clear stored samples

Notes

- None

3.4.2 GetSampleCount

Method Signature

- `int _stdcall GetSampleCount();`

Description
- Returns the number of samples stored on the instrument

Input Parameters
- None

Return Value
- The number of samples stored on the instrument

Notes
- None

3.4.3 SetCurrentSample

Method Signature
- `BOOL _stdcall SetCurrentSample(int sample);`

Description
- Sets the current sample, based on the index provided (zero based)

Input Parameters
- sample – index of desired sample

Return Value
- true – succeeded in setting the current sample
- false – failed to set current sample

Notes
- This method *must be* called before calling GetSampleData().

3.4.4 GetSampleData

Method Signature
- `float _stdcall GetSampleData(int spectralset, int wl);`

Description
- Returns spectral data for a given spectral and wavelength for the selected stored sample

Input Parameters
- spectralset – index of the desired spectral set (zero based)
- wl – index of the desired wavelength (zero based)

Return Value

- Spectral data for the given spectral set and wavelength

Notes
- This method is similar to GetSpectralData(), except it returns spectral data for the currently selected stored sample, not for the last measurement.
- The current sample is specified using the SetCurrentSample() method.
- 100% reflectance is returned as 100.00

3.5 Settings

3.5.1 GetAvailableSettings

Method Signature
- `const char* _stdcall GetAvailableSettings();`

Description
- Get the list of available settings for the instrument

Input Parameters
- None

Return Value
- The list of settings for the instrument, formatted as a semi-colon delimited string

Notes
- See the table in section "3.5.5 Settings Table" for details for each instrument.
- Note that items marked n/a in the Settings Table are not returned as available settings.
- Example for Ci51: return value is "Connection_Method;Reading_Mode;NetProfiler"
- Example for RM400: return value is "Connection_Method;Reading_Mode" "

3.5.2 GetSettingOptions

Method Signature
- `const char* _stdcall GetSettingOptions(const char* setting);`

Description
- Returns the list of options (valid values) for the given setting

Input Parameters
- setting – name of the setting for which the options are desired

Return Value
- The list of options for the given setting, formatted as a semi-colon delimited string
- If an invalid setting is provided, the return value will be " "

Notes

- See the table in section “3.5.5 Settings Table” for details for each instrument.
- Note that items marked n/a in the Settings Table are not returned as available settings.
- Example for Ci51 for Reading_Mode: return value is “Pressure;Button;Pressure_and_Button;Software”
- The options for connection method depend on the available ports on the computer. Where the table indicates “COM#” as a connection method, the driver will return the list of available COM ports on the computer, e.g. “COM1;COM2;COM7”.
- Example for RM400 for Connection_Method: return value is “COM1;COM2”, assuming the computer has COM1 and COM2 ports.

3.5.3 GetOption

Method Signature

- `const char* _stdcall GetOption(const char* setting);`

Description

- Get the current option for a setting

Input Parameters

- setting – name of the setting whose current option is desired

Return Value

- The current option for the given setting
- If an invalid setting is provided, the return value will be “ “

Notes

- See the table in section “3.5.5 Settings Table” for details for each instrument.
- Example for Ci51 for Reading_Mode: return value could be “Pressure”

3.5.4 SetOption

Method Signature

- `BOOL _stdcall SetOption(const char* setting, const char* option);`

Description

- Set the current option for a setting

Input Parameters

- setting – the setting whose option will be changed
- option – the option to set for the setting

Return Value

- true – successfully set the option
- false – failed to set the option

Notes

- See the table in section "3.5.5 Settings Table" for details for each instrument.
- Example for Ci51: result = SetOption("Reading_Mode", "Pressure");
- The drivers will store the option in a persistent manner, so the calling application does not need to store settings.
- All settings can be changed before or after Connect() is called.
- If the Connection_Method setting is not set before Connect() is called, it will default to the last option set by the application. If the application has never set the Connection_Method, it will default as indicated in section 3.5.5, "Settings Table".

3.5.5 Settings Table

Notes

- * indicates the default option
- If NetProfiler is set to On, but no profile is available, it is effectively set to Off.
- For Connection_Method, the default COM port is the first COM port reported by the system.
- For all instruments, no matter which Reading_Mode is set, a measurement can be initiated via software. In other words, the driver behaves as if the Software reading mode is always set in addition to the current reading mode.
- When Sample_Storage is set to On, the device is set into storage mode. This means subsequent measurements will be stored in the device, regardless of whether the instrument is connected to the computer.

Device	Measurement_Area	Reading_Mode	NetProfiler	Connection_Method	Sample_Storage	Measurement_Mode
552	n/a	Pressure	n/a	COM#	n/a	n/a
939	n/a (software cannot determine or set)	Pressure* Button Pressure_and_Button Software	n/a	COM#	On, Off*	n/a
962	n/a (software cannot determine or set)	Pressure* Button Pressure_and_Button Software	On* Off	COM#	On, Off*	n/a
964	n/a (software cannot determine or set)	Pressure* Button Pressure_and_Button Software	On* Off	COM#	On, Off*	n/a
CF57	n/a	Software	n/a	COM#	n/a	n/a
CFS57	n/a	Software	n/a	COM#	n/a	n/a

Ci4100	Medium	Button* Software	n/a	USB	n/a	n/a
Ci4200	Medium	Button* Software	n/a	USB	n/a	n/a
Ci4200UV	Medium	Button* Software	n/a	USB	n/a	Tungsten UV_D65 UV_ADJ1 UV_ADJ2
Ci5	Auto* Small Medium Large	Button* Software	On* Off	COM#	n/a	n/a
Ci51	Medium	Pressure* Button Pressure_and_Button Software	On* Off	USB	n/a	n/a
Ci52	Medium	Pressure* Button Pressure_and_Button Software	On* Off	USB	n/a	n/a
Ci61	Medium	Pressure* Button Pressure_and_Button Software	On* Off	USB* BluetoothName	On* Off	n/a
Ci61+RTL	Medium	Pressure* Button Pressure_and_Button Software	On* Off	USB* BluetoothName	On* Off	n/a
Ci62/Ci62S	Small Medium	Pressure* Button Pressure_and_Button Software	On* Off	USB* BluetoothName	On Off*	n/a
Ci64/Ci64L	Small Medium Large	Pressure* Button Pressure_and_Button Software	On* Off	USB* BluetoothName	On Off*	n/a
Ci64UV	Small Medium	Pressure* Button Pressure_and_Button Software	On* Off	USB* BluetoothName	On Off*	Tungsten UV_D65 UV_ADJ1 UV_ADJ2
Ci7	Auto* Small Medium Large 17 mm	Button* Software	On* Off	COM#	n/a	n/a
eXact	1.5mm 2mm 4mm	Pressure* Button, Software	On* Off	USB* BluetoothName	n/a	n/a
MA96	Large	Pressure* Button Pressure_and_Button Software	n/a	USB* BluetoothName	On* Off	n/a
RM400	n/a	Button Software	n/a	COM#	n/a	n/a
RM425	n/a	Button	n/a	COM#	n/a	n/a
RM61	n/a	Pressure	n/a	COM#	n/a	n/a

SP52	n/a	Pressure	n/a	COM#	n/a	n/a
SP62	n/a (software cannot determine or set)	Pressure* Button Pressure_and_Button Software	On* Off	COM#	On, Off*	n/a
SP64	n/a (software cannot determine or set)	Pressure* Button Pressure_and_Button Software	On* Off	COM#	On, Off*	n/a
VS205	Small Large*	Software	n/a	USB	n/a	Single_Lamp Dual_Lamp
VS450	Small Large*	Software	n/a	USB	n/a	Single_Lamp Dual_Lamp

3.6 Scanning Measurement

These methods are new to XDS4 version 1.2 and not available in previous versions.

3.6.1 ScanIsSupported

Method Signature

- `BOOL _stdcall ScanIsSupported();`

Description

- Determines whether scan measurements are supported for this instrument.

Input Parameters

- None

Return Value

- true – instrument supports scan measurements.
- false – instrument does not support scan measurements.

Notes

- None

3.6.2 ScanConfig

Method Signature

- `BOOL _stdcall ScanConfig(int patchCount, float patchWidth);`

Description

- Configures the instrument with required information about color bar that is to be scanned.

Input Parameters

- patchCount – the maximum number of patches in color bar to be measured.

- patchWidth – the width (in millimeters) individual patches within color bar. All patches within color bar must be same width.

Return Value
- true – successfully configured scan parameters.
- false – failed to configure scan parameters.

Notes
- This method is required to be called prior to calling ScanStart method.

3.6.3 ScanStart

Method Signature
- `BOOL _stdcall ScanStart();`

Description
- Tells the driver to place instruments into scanning mode.

Input Parameters
- None

Return Value
- true – scanning mode was properly initiated.
- false – scanning mode was not initiated

Notes
- This ScanConfig method is required to be called at least once during a connection prior to calling this method.
- User should call the ScanGetStatus method to determine when scanning measurement is completed.
- Calling this method will clear any previously measured scanned data.

3.6.4 ScanAbort

Method Signature
- `BOOL _stdcall ScanAbort();`

Description
- Tells the driver to abort a scanning measurement

Input Parameters
- None

Return Value
- true – measurement was aborted

- false – measurement was not aborted successfully

Notes

- This ScanStart method should have already been called to initiate a measurement prior to this method call.
- After this method is called, the ScanGetStatus will return a READYTOSCAN status.

3.6.5 ScanGetStatus

Method Signature

- `int _stdcall ScanGetStatus();`

Description

- Indicates the status of a scanning measurement.

Input Parameters

- None

Return Value

- int represented by the following enum.

```
enum SCANSTATUS
    {
        SCANERROR = -1,
        IDLE = 0,                   // ScanStart has not been called.
        READYTOSCAN = 1,            // ScanStart
        SCANINPROGRESS = 2,
        SCANCOMPLETE = 3,
        PAPERINPROGRESS = 4,
        PAPERCOMPLETE = 5
    };
```

  - SCANERROR – Error occurred during scanning measurement.
  - IDLE – ScanStart has not yet been called. Instrument not in scanning mode.
  - READYTOSCAN – ScanStart called and waiting on user to perform scanning measurement.
  - SCANINPROGRESS – Scanning measurement is in progress.
  - SCANCOMPLETE – Scanning measurement completed.
    - The ScanGetStatus will only return this status once when a scanning measurement is completed. It will then return to a READYTOSCAN status.
  - PAPERINPROGRESS – Paper measurement in progress.
  - PAPERCOMPLETE – Paper measurement successfully completed.
    - The ScanGetStatus will only return this status once when a paper measurement is completed. It will then return to a READYTOSCAN status.

Notes

- Once the ScanStart method is called, the application should poll the driver using this command, to determine when the scanning measurement is complete and the data is ready.
- The SCANCOMPLETE status (5) result will only be returned once after a successful scan. The status is then automatically cleared and instrument returns to READYTOSCAN status. However, the scan data will remain available.

3.6.6 ScanGetCount

Method Signature

- `int _stdcall ScanGetCount();`

Description

- Returns the number of patches measured by the last successful scanning measurement.

Input Parameters

- None

Return Value

- The number of patches measured by the last scanning measurement. The data is available using the ScanGetData method.

Notes

- This method should be called after scanning measurement is completed and ScanGetStatus returns the SCANCOMPLETE status.

3.6.7 ScanGetData

Method Signature

- `const char* _stdcall ScanGetData(int patchIndex, int dataSet);`

Description

- Returns spectral data for a specified patch measured using a scanning measurement.

Input Parameters

- patchIndex – index of the desired patch (zero based)
- dataSet – index of the desired spectral set (zero based)

Return Value

- A space separated string that contains the list of 31 reflectance values for measured patch. 100% reflectance is returned as 100.00

- There will always be 31 values representing the reflectance values from 400nm to 700nm in 10nm increments.
- The reflectance values always use a '.' symbol as the Decimal symbol no matter the current culture.

Notes

- None

3.7 Error Handling

3.7.1 GetLastErrorCode

Method Signature

- `int _stdcall GetLastErrorCode();`

Description

- Gets code of last error. See Section 4, "Error Codes" for a list of error codes.

Input Parameters

- None

Return Value

- Numeric code of last error

Notes

- Error codes may represent a problem with the instrument, driver, or communication interface.

3.7.2 GetLastErrorString

Method Signature

- `const char* _stdcall GetLastErrorString();`

Description

- Gets textual description of last error

Input Parameters

- None

Return Value

- Textual description of last error

Notes

- Error strings are published in Section 4, "Error Codes"
- Error strings are intended for the application developer, not for the end user of the application.
- Error strings are available in English only.

3.8 Pass-Through or Custom Commands

3.8.1 Execute

Method Signature

- `const char* _stdcall Execute(const char* cmdString);`

Description

- Sends a custom command to the driver or a pass-through command to the instrument. If the command is not recognized by the driver as a custom command, it will be passed on to the instrument as-is. Custom commands are not common and they vary by instrument.

Input Parameters

- cmdString – command to send to instrument; format is "command;arg1;arg2;..;argn"

Return Value

- If return string is empty, use GetLastErrorCode() and GetLastErrorString() to discover what error occurred

Notes

- For information about commands for a particular instrument, see the Command Specification document for that instrument.

3.8.2 ExecuteBinary

Method Signature

- `long _stdcall ExecuteBinary(const char* inputBuffer, long inputLength, long timeout, char* outputBuffer, long outputBufferSize);`

Description

- Sends a custom command to the driver or a pass-through command to the instrument. Similar to "Execute", but allowing for binary data to be sent and/or received. The caller is responsible for all memory management.

Input Parameters

- inputBuffer – Contents of binary data sent to device.
- inputLength – Size of inputBuffer parameter in bytes.
- timeout – Timeout in milliseconds for command response before command before a RECEIVE_TIMEOUT (0x5005) error code is generated.

- outputBuffer – Buffer where response will be written to. Caller is responsible for properly managing memory.
- outputBufferSize – Size of outputBuffer in bytes that has been allocated. If response is larger than buffer size, then the received data will be truncated to outputBufferSize.

Return Value

- Number of bytes received. If an error occurs, then 0 will be returned.

Notes

- For information about commands for a particular instrument, see the Command Specification document for that instrument.
- This is a straight pass through command and there is no error checking or manipulation of data within the XDS4 driver.
- This method is new to XDS4 version 1.2 and not available in previous versions.

4 Error Codes

Notes

- The last error code can be retrieved using the GetLastErrorCode() method.
- The last error string can be retrieved using the GetLastErrorString() method.

4.1.1 General Errors

	Error Code (hex)	Error String
DEVICE_NOT_FOUND	20480 (0x5000)	Device not found
SEND_NOT_CONNECTED	20481 (0x5001)	Send: Not Connected
SEND_TIMEOUT	20482 (0x5002)	Send: Timeout
SEND_FAILED_WRITE	20483 (0x5003)	Send: Failed to Write Data
RECEIVE_NOT_CONNECTED	20484 (0x5004)	Receive: Not Connected
RECEIVE_TIMEOUT	20485 (0x5005)	Receive: Timeout
RECEIVE_PACKET_LENGTH	20486 (0x5006)	Receive: Invalid packet length
RECEIVE_BAD_BCP	20487 (0x5007)	Receive: Invalid BCP
INCORRECT_MODEL	20488 (0x5008)	Incorrect Instrument Model
MEASURE_NEEDS_CAL	20489 (0x5009)	Measure: Needs Calibration
MEASURE_TIMEOUT	20490 (0x500A)	Measure: Timeout
CALIBRATION_ERROR	20491 (0x500B)	Calibration: Failed
CALIBRATION_NOT_CONNECTED	20492 (0x500C)	Calibration: Not Connected
CALIBRATION_IN_PROGRESS	20493 (0x500D)	Calibration already in progress
CALIBRATION_INVALID_STEP	20494 (0x500E)	Calibration: Invalid Step
CALIBRATION_ABORTED	20495 (0x500F)	Calibration: User Aborted
INVALID_SETTING	20496 (0x5010)	Invalid Setting
INVALID_OPTION	20497 (0x5011)	Invalid Option
LOCKPORT_FAILED	20498 (0x5012)	Communications Port is busy
INVALID_MEASUREMENT	20499 (0x5013)	Invalid Measurement
INVALID_SAMPLE	20500 (0x5014)	Invalid Sample
CONFIGURE_ABORT	20501 (0x5015)	Configure: User Aborted
OPERATION_NOT_SUPPORTED	20502 (0x5016)	Operation Not Supported

4.1.2 Instrument Specific Errors

Errors specific to a particular instrument are described in the supplemental XDS 4 document for that instrument.